

SECCIÓN 1: INVESTIGACIÓN TEÓRICA

1. Control de Versiones

Definición: Es un sistema que registra los cambios realizados sobre archivos a lo largo del tiempo, de modo que sea posible recuperar versiones anteriores cuando sea necesario.

Utilidad: Cuando varias personas trabajan sobre el mismo proyecto, el control de versiones evita que se pisen el trabajo unos a otros. También permite deshacer errores y ver exactamente qué cambió, cuándo y quién lo hizo.

2. Git

Definición: Git es un sistema de control de versiones distribuido, de código abierto y gratuito, diseñado para manejar desde proyectos pequeños hasta muy grandes con velocidad y eficiencia. Fue creado por Linus Torvalds en 2005.

Utilidad: Con Git puedes trabajar sin internet, crear ramas para probar ideas sin romper nada, fusionar cambios de varios colaboradores y mantener un historial limpio de todo lo que ha ocurrido en el proyecto.

3. GitHub

Definición: GitHub es una plataforma de alojamiento de código basada en la nube que utiliza Git para el control de versiones. Además del alojamiento de repositorios, ofrece herramientas de colaboración como Issues, Pull Requests, Actions y Projects.

Utilidad: Permite que equipos distribuidos trabajen juntos, revisar el código de otros antes de integrar cambios, automatizar pruebas y despliegues, y construir un portafolio profesional visible para empleadores.

4. Repositorio Local

Definición: Un repositorio local es la copia del proyecto almacenada en la máquina del desarrollador. Contiene todos los archivos del proyecto, el historial completo de commits y las ramas de trabajo, todo almacenado en la carpeta oculta .git.

Utilidad: Al tener el repositorio en local, puedes hacer commits, revisar el historial y crear ramas sin necesidad de estar conectado a internet. Sincronizas con el servidor solo cuando lo necesitas.

5. Repositorio Remoto

Definición: Un repositorio remoto es una versión del proyecto alojada en un servidor accesible a través de internet o una red. Actúa como punto central de colaboración donde todos los integrantes del equipo sincronizan sus cambios.

Utilidad: Sirve como respaldo del proyecto y como punto de encuentro donde todos los desarrolladores sincronizan su trabajo. Sin él, coordinar a un equipo distribuido sería prácticamente imposible.

6. Commit

Definición: Un commit es una instantánea del estado del proyecto en un momento determinado. Representa un conjunto de cambios confirmados en el repositorio local, acompañado de un mensaje descriptivo, el autor y una marca de tiempo.

Utilidad: Es la unidad básica de trabajo en Git. Gracias a los commits puedes ver qué se cambió en cada momento, volver atrás si algo sale mal, y entender la evolución del proyecto con el tiempo.

7. Branch (Rama)

Definición: Una rama es una línea independiente de desarrollo dentro de un repositorio. Permite trabajar en nuevas funcionalidades, correcciones de errores o experimentos sin afectar el código principal (usualmente la rama main o master).

Utilidad: Las ramas hacen posible el desarrollo paralelo: cada integrante trabaja en la suya sin molestar al otro. Solo cuando el trabajo está listo y revisado se integra al código principal.

8. Merge

Definición: El merge (fusión) es la operación que integra los cambios de una rama hacia otra. Git analiza el historial de ambas ramas y combina los cambios, creando un commit de fusión que une los dos

historiales.

Utilidad: Es el mecanismo principal para integrar trabajo completado al código principal. Permite consolidar funcionalidades desarrolladas en paralelo y es el paso final del flujo Pull Request en GitHub.

9. Pull Request

Definición: Un Pull Request (PR) es una solicitud formal para integrar los cambios de una rama hacia otra dentro de GitHub. No es una característica de Git en sí, sino una funcionalidad de GitHub que facilita la revisión y discusión de cambios antes de integrarlos.

Utilidad: Permite la revisión de código por pares, facilita la discusión sobre los cambios propuestos, muestra las diferencias entre ramas, verifica que las pruebas automatizadas pasen y proporciona un historial auditado de decisiones.

10. Fork

Definición: Un fork es una copia personal de un repositorio de otro usuario en la propia cuenta de GitHub. El fork es completamente independiente del repositorio original pero mantiene una referencia a él para poder sincronizar cambios.

Utilidad: Permite contribuir a proyectos en los que no se tiene acceso de escritura directa, es la base del modelo de colaboración en proyectos de código abierto, y permite experimentar libremente sin afectar el proyecto original.

11. Conflictos de Integración

Definición: Un conflicto de integración ocurre cuando dos ramas han modificado la misma línea de un archivo de forma diferente, y Git no puede determinar automáticamente cuál versión conservar durante un merge. Git marca las secciones conflictivas en el archivo.

Utilidad: Comprender los conflictos es esencial para el trabajo en equipo. Su resolución manual asegura que la lógica combinada del equipo sea correcta. Git proporciona marcadores especiales (<<<<<<, =====, >>>>>>) para identificarlos.

12. Integración Continua (CI)

Definición: La Integración Continua es una práctica de desarrollo donde los miembros del equipo integran su trabajo frecuentemente (al menos una vez al día). Cada integración se verifica mediante una build automatizada con pruebas para detectar errores rápidamente.

Utilidad: Reduce los problemas de integración, detecta bugs de forma temprana, proporciona retroalimentación rápida, mejora la calidad del código y permite mantener el software en un estado siempre desplegable.

13. Entrega Continua (Continuous Delivery)

Definición: La Entrega Continua es una práctica donde el software es construido, probado y preparado de forma automática para su liberación a producción. Extiende la CI asegurando que el código siempre esté en un estado desplegable, aunque el despliegue final puede requerir aprobación manual.

Utilidad: Reduce el riesgo de los lanzamientos de software, acorta el ciclo de retroalimentación con los usuarios, permite lanzamientos más frecuentes y confiables, y alinea al equipo técnico con los objetivos de negocio.

14. Despliegue Continuo (Continuous Deployment)

Definición: El Despliegue Continuo lleva la Entrega Continua un paso más allá: cada cambio que pasa todas las etapas del pipeline se despliega automáticamente a producción sin intervención humana. Requiere una alta confianza en los procesos automatizados.

Utilidad: Elimina el proceso manual de despliegue, acelera la entrega de valor a los usuarios, permite múltiples despliegues diarios, reduce el riesgo al hacer cambios pequeños y frecuentes, y mejora la respuesta ante problemas.

15. GitHub Actions

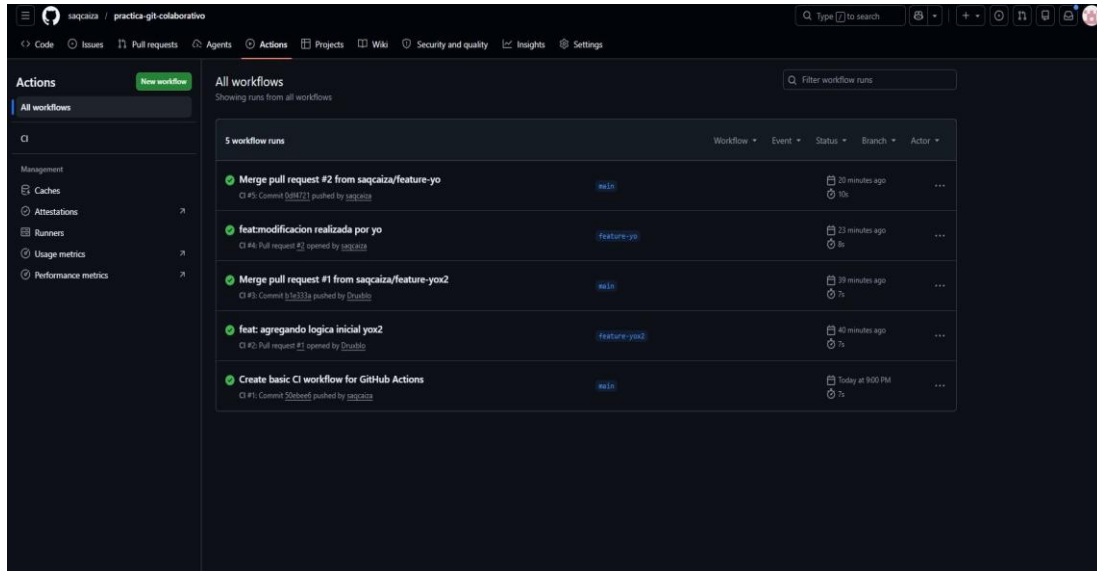
Definición: GitHub Actions es la plataforma de automatización y CI/CD integrada en GitHub. Permite crear flujos de trabajo (workflows) definidos en archivos YAML que se ejecutan automáticamente ante eventos del repositorio como push, pull requests o schedules.

Utilidad: Automatiza tareas repetitivas como ejecutar pruebas, verificar estilo de código, construir artefactos y desplegar aplicaciones. Está integrado nativamente con GitHub

SECCIÓN 2: DEMOSTRACIÓN PRÁCTICA

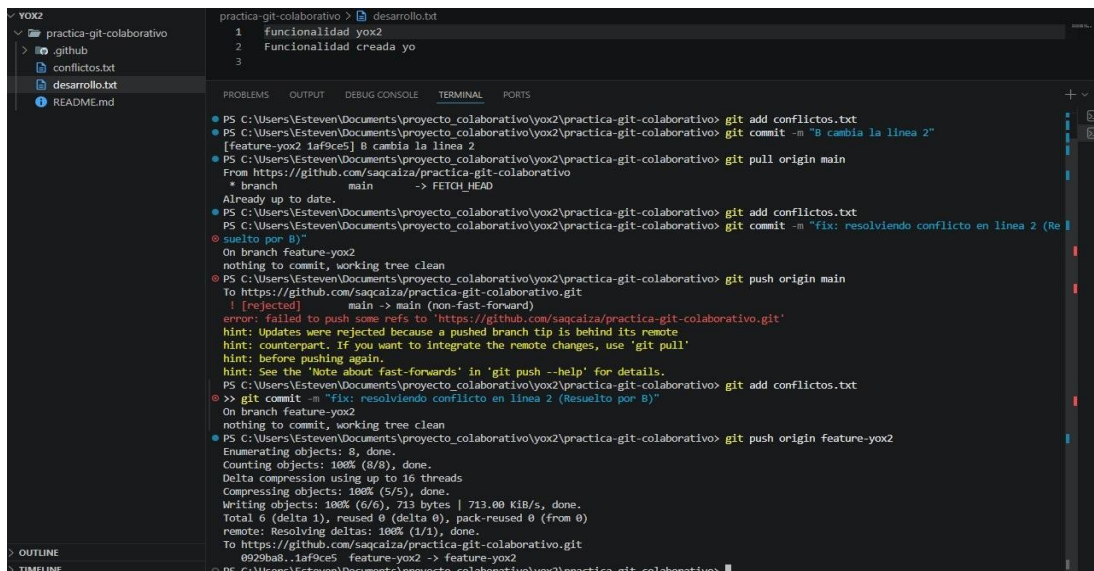
Paso 1: Creación del Repositorio en GitHub

Steeven (saqcaiza) creó el repositorio público 'practica-git-colaborativo' en su cuenta de GitHub. El repositorio fue inicializado con un archivo README.md. En esta etapa el repositorio cuenta con 1 commit inicial y se encuentra en la rama main. La URL del repositorio es: <https://github.com/saqcaiza/practica-git-colaborativo.git>



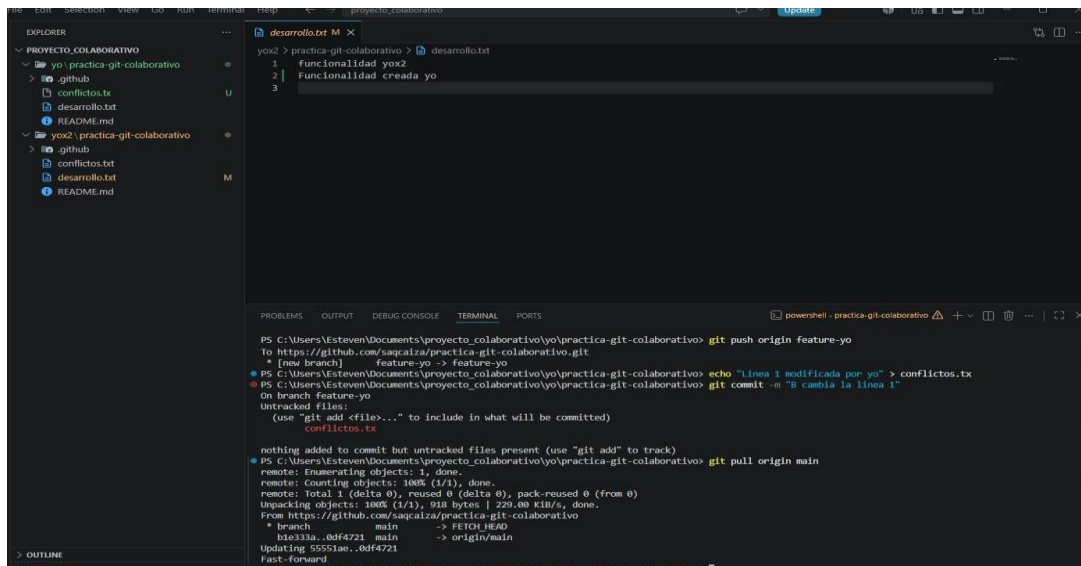
Paso 2: Configuración del Workflow de GitHub Actions (CI)

El Estudiante A configuró el workflow de Integración Continua directamente desde la interfaz web de GitHub. Se creó el archivo .github/workflows/blank.yml con un pipeline básico de CI que se activa en eventos de push y pull_request hacia la rama main. El commit del workflow lleva el mensaje descriptivo 'Create basic CI workflow for GitHub Actions'.



Paso 3: Rama y Commits del Estudiante B (Druxblo)

El Estudiante B (Druxblo) clonó el repositorio en su máquina local. Creó la rama 'feature-yox2', añadió el archivo desarrollo.txt con el contenido inicial y realizó el commit con el mensaje 'feat: agregando logica inicial yox2'. Inicialmente hubo un error 403 por credenciales, que fue resuelto configurando correctamente el token de acceso personal.



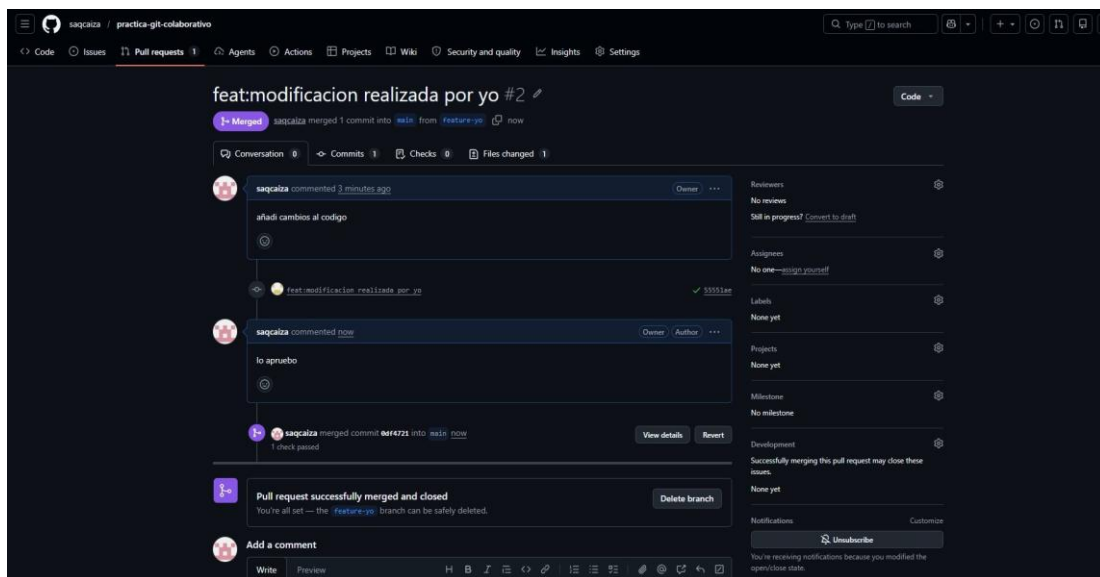
The screenshot shows a VS Code editor with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project named 'PROYECTO_COLABORATIVO' with subfolders 'yo' and 'yox2'. The terminal window displays the following commands and output:

```
PS C:\Users\Esteven\Documents\proyecto_colaborativo\yo\practica-git-colaborativo> git push origin feature-yo
To https://github.com/saqcaiza/practica-git-colaborativo.git
 * [new branch]   feature-yo -> feature-yo
PS C:\Users\Esteven\Documents\proyecto_colaborativo\yo\practica-git-colaborativo> echo "línea 1 modificada por yo" > conflictos.tx
PS C:\Users\Esteven\Documents\proyecto_colaborativo\yo\practica-git-colaborativo> git commit -m "B cambia la línea 1"
On branch feature-yo
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  conflictos.tx

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\Esteven\Documents\proyecto_colaborativo\yo\practica-git-colaborativo> git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 918 bytes | 229.00 KiB/s, done.
From https://github.com/saqcaiza/practica-git-colaborativo
 * branch            main       -> FETCH_HEAD
   b1e133a..0df4721  main       -> origin/main
Updating 55551ae..0df4721
Fast-forward
PS C:\Users\Esteven\Documents\proyecto_colaborativo\yo\practica-git-colaborativo>
```

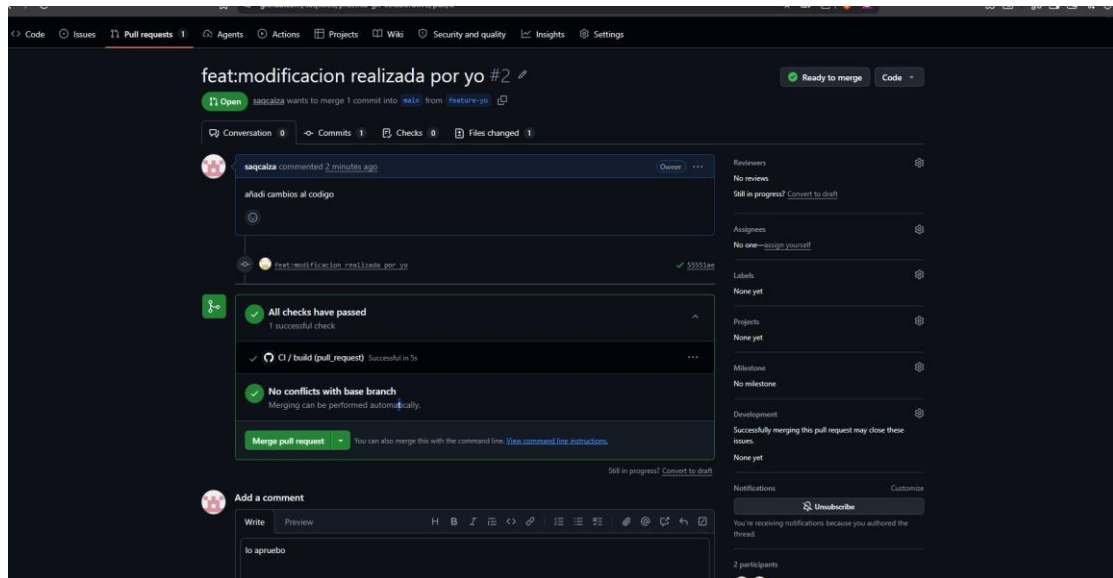
Paso 4: Creación del Pull Request #1 por Druxblo

Una vez publicada la rama 'feature-yox2', Druxblo procedió a crear el Pull Request #1 desde la interfaz de GitHub en español. Se seleccionó como base la rama 'principal' (main) y como rama a comparar 'característica-yox2'. El PR fue titulado 'feat: agregando logica inicial yox2', siguiendo la convención de Commits Convencionales.



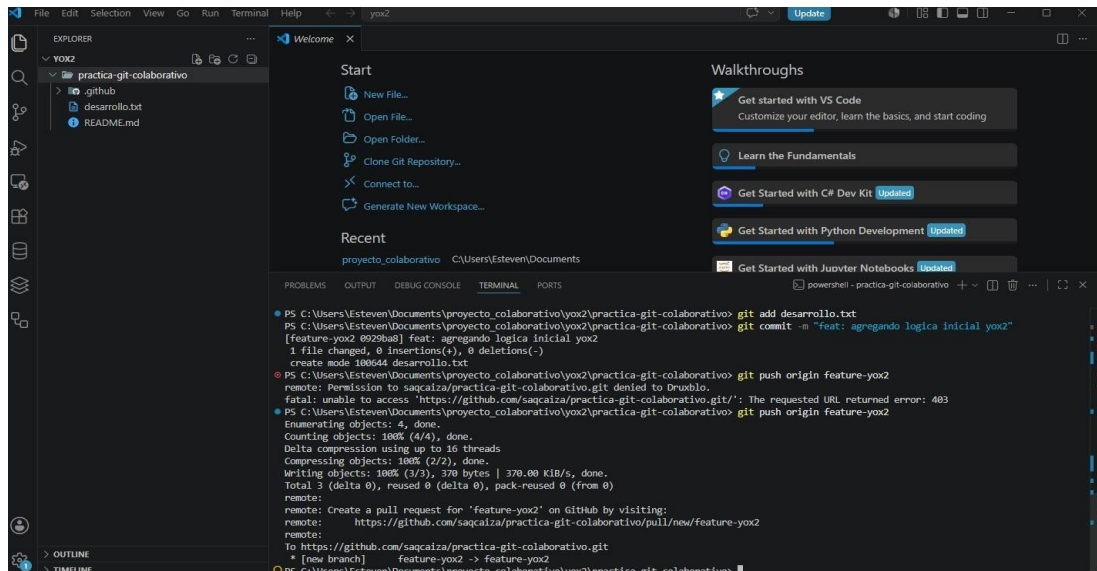
Paso 5: Revisión y Merge del Pull Request #1 por saqcaiza

El Estudiante A (saqcaiza) revisó el Pull Request #1. Tras verificar los cambios propuestos, aprobó y ejecutó el merge de 'feature-yox2' hacia 'main'. El PR muestra estado 'Merged' con etiqueta de colaborador para Druxblo. El merge fue exitoso sin conflictos.



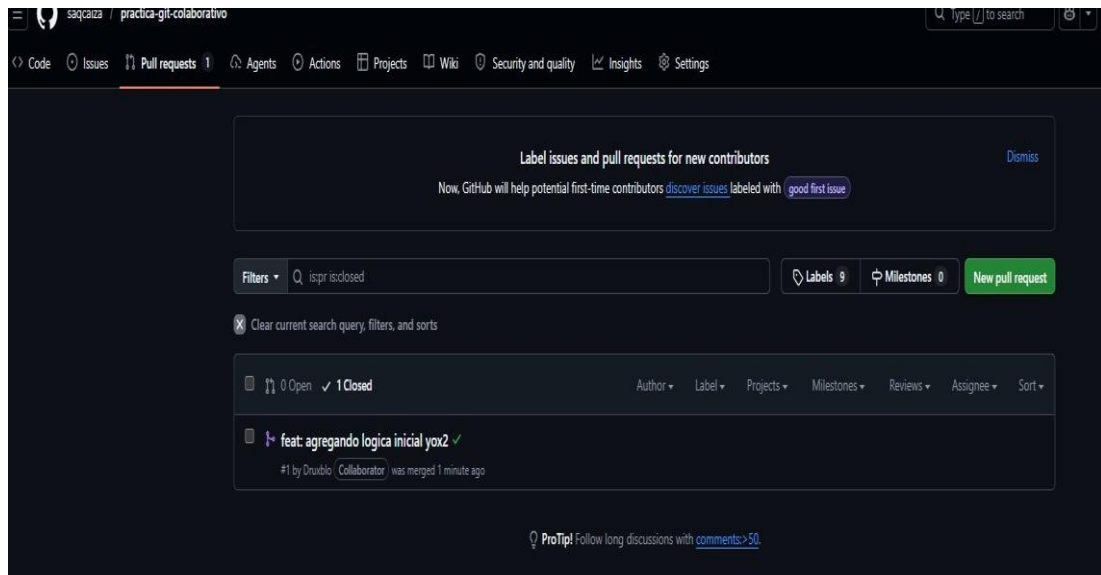
Paso 6: Intercambio de Roles — Rama y Commits de saqcaiza

Intercambiando roles, saqcaiza actuó como desarrollador colaborador. Creó la rama 'feature-yo' en su copia local. Añadió contenido al archivo conflictos.txt con el texto 'Linea 1 modificada por yo' y realizó el pull de los últimos cambios de main para estar actualizado antes de generar el PR.



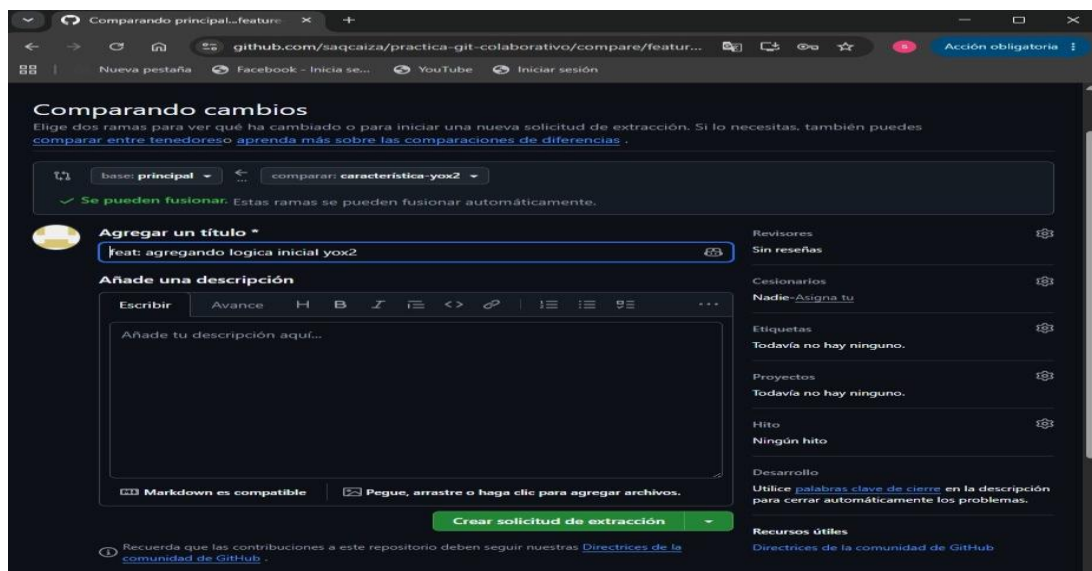
Paso 7: Pull Request #2 — Revisión, CI y Aprobación

saqcaiza creó el Pull Request #2 'feat.modificacion realizada por yo' desde 'feature-yo' hacia 'main'. GitHub Actions ejecutó automáticamente el workflow CI. El resultado fue positivo: 'All checks have passed' con el check 'CI / build (pull_request) Successful in 5s'. También se verificó que no había conflictos con la rama base: 'No conflicts with base branch'. El PR quedó listo para merge.



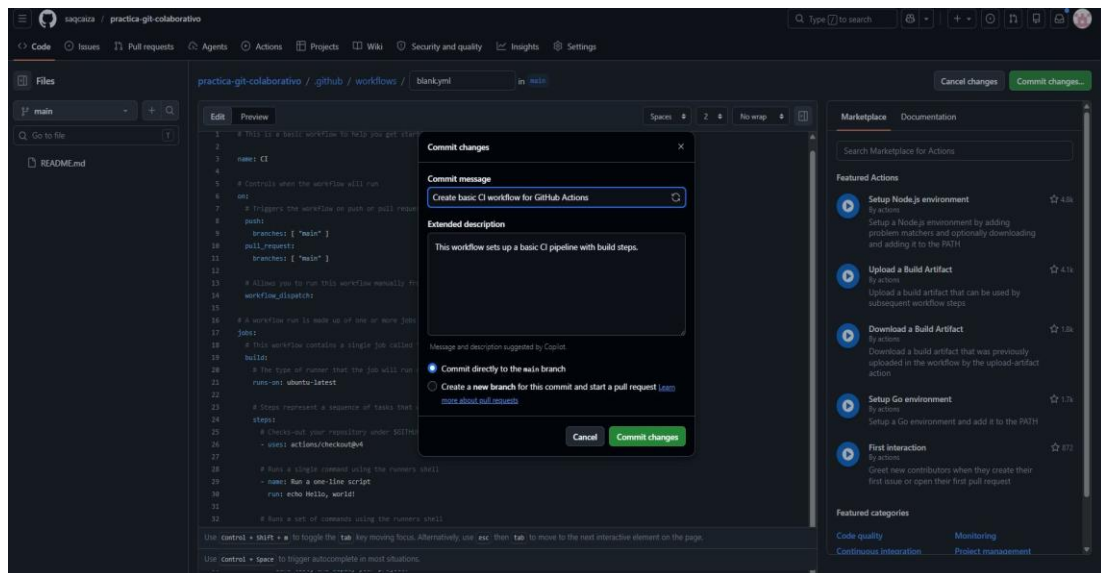
Paso 8: Merge del Pull Request #2

El PR #2 fue mergeado a main. saqcaiza, actuando como revisor, añadió el comentario 'lo apruebo' y ejecutó el merge del commit 0df4721. Se muestra el estado 'Pull request successfully merged and closed' y la opción de eliminar la rama feature-yo de forma segura.



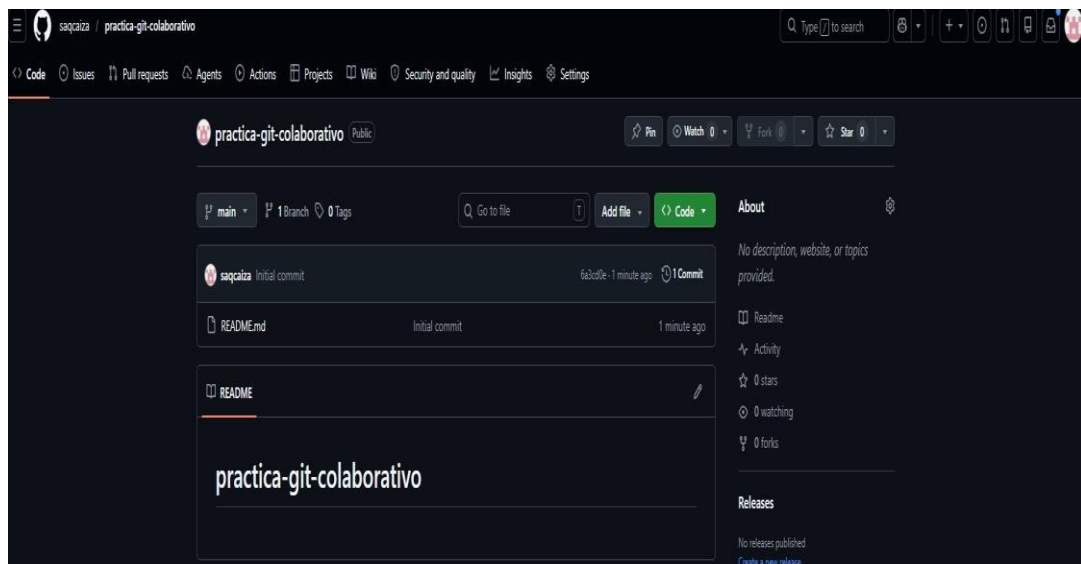
Paso 9: Generación y Resolución de Conflicto de Integración

Para demostrar la resolución de conflictos, ambos integrantes modificaron la misma línea del archivo conflictos.txt en sus respectivas ramas. Druxblo modificó la línea 2 ('B cambia la línea 2') y saqcaiza también modificó esa misma línea. Al intentar hacer push a main, el sistema rechazó el push con error 'non-fast-forward'. El equipo resolvió el conflicto manualmente eligiendo la versión correcta y realizando el commit 'fix: resolviendo conflicto en línea 2 (Resuelto por B)'. Finalmente se hizo push a la rama feature-yox2 exitosamente.



Paso 10: Ejecución Exitosa del Workflow de GitHub Actions

La pestaña Actions del repositorio muestra el historial completo de los 5 workflows ejecutados durante la práctica. Todos los runs muestran estado exitoso (check verde). Los runs incluyen: el workflow inicial creado directamente en main, los runs disparados por los PRs de ambas ramas (feature-yox2 y feature-yo), y los merges finales hacia main. Los tiempos de ejecución van de 7 a 10 segundos.



CONCLUSION

Conclusion

Durante el desarrollo de esta práctica pude entender de verdad por qué el control de versiones es tan importante en el trabajo en equipo. No es solo guardar archivos: es saber exactamente qué cambió, quién lo cambió y por qué. Alternar entre el rol de administrador del repositorio y el de desarrollador colaborador me permitió ver el proceso desde ambos lados, lo cual creo que es clave para entenderlo bien. Configurar el pipeline de GitHub Actions fue lo que más me gustó, porque en cada Pull Request podía ver en tiempo real si el código estaba bien o no, sin revisar manualmente nada. El momento más difícil fue sin duda la resolución del conflicto de integración: entender por qué Git rechazaba el push y saber qué líneas conservar requirió leer los mensajes con cuidado y tomar decisiones en conjunto. Me quedo con la idea de que Git y GitHub, bien usados, le dan estructura y seguridad al trabajo colaborativo de una forma que no se puede reemplazar fácilmente.
